实验报告02：NumPy、SciPy、Matplotlib 与 pandas 数据处理

一、实验环境与提交要求

1. 请使用独立的 conda 虚拟环境完成实验，不得使用 `base` 环境。在该环境中准备 Python 3、Jupyter Notebook、NumPy、SciPy、Matplotlib 和 pandas。
2. 请先将第一个代码框中的学号修改为自己的学号，再按顺序运行各代码框。可在 PyCharm 或 VS Code 中完成。
3. 使用 Markdown 单元格解释关键结果；使用代码单元格完成计算。图表须有标题、坐标轴标签和必要的图例。
4. 涉及随机抽样的实验请设置固定种子。
5. 提交前重启内核并运行全部代码，检查依赖和输出。只提交命名为 `学号_姓名_exp02.ipynb` 的 Notebook。

1.1 第一个代码框：身份与环境记录

```
import os
import uuid
import hashlib
import platform
import numpy as np
import scipy
import matplotlib
import pandas as pd

# 修改为自己的学号
STUDENT_ID = 20190010

CONDA_ENV = os.environ.get("CONDA_DEFAULT_ENV", "base")
print(f"[Conda 环境] {CONDA_ENV}")

MACHINE_FP = hashlib.sha256(
    f"{platform.node()}|{platform.machine()}|{platform.system()}|
{uuid.getnode()}".encode()
).hexdigest()[:16]
RUN_ID = f"{STUDENT_ID}-{MACHINE_FP}"
print(f"[学号]       {STUDENT_ID}")
print(f"[机器指纹] {MACHINE_FP}")
print(f"[RUN_ID]   {RUN_ID}")
```

二、实验内容与过程

实验1：NumPy 数组、数据类型与索引

操作任务：

1. 分别创建整数、浮点数和布尔值 `ndarray`，输出每个数组的 `dtype`、`shape` 和 `ndim`。解释 NumPy 数组的数据类型与 Python 列表的差别。
2. 用 `np.arange(12)` 创建一维数组，并分别变为形状 `(3, 4)` 和 `(2, 3, 2)` 的数组。展示数组内容，说明改变形状时元素总数必须满足什么条件。
3. 对形状为 `(3, 4)` 的数组完成以下索引：取第 2 行、取第 1 至第 3 列、取大于 6 的元素，并分别输出结果及其形状。
4. 使用 `np.arange(10)` 分别选出“小于 2 或大于 7”和“大于 3 且小于 7”的元素。使用括号配合 `|`、`&` 或 `np.logical_or()`、`np.logical_and()`。
5. 构造两个形状为 `(2, 2)` 的数组，对比 `*`、`np.multiply()` 与 `@`、`np.matmul()` 的结果，说明逐元素乘法与矩阵乘法的区别。

实验2：随机抽样、统计与缺失值

操作任务：

1. 使用 `np.random` 生成 1,000 个均值为 70、标准差为 10 的模拟成绩；用`np.clip` 将数值限制在 0～100 范围内。输出样本数、最小值、最大值、均值、标准差和极差。
2. 从 `np.arange(1, 11)` 中不放回地抽取 5 个整数，比较均匀抽样与按整数大小成比例加权抽样的写法。
3. 分别演示随机排列后**保留原数组**和**原地打乱数组**，打印操作前后的原数组以说明两种方式的区别。
4. 将模拟成绩的第 10、20、30 个值设为 `np.nan`。比较普通均值和忽略缺失值后的均值，统计缺失值个数，并说明 `nan` 对普通统计函数的影响。
5. 将含缺失值的模拟成绩及其序号整理成内存中的表格变量，供实验4直接使用；不要写入磁盘。

实验3：SciPy 数值计算与 Matplotlib 可视化

操作任务：

1. 使用 `scipy.linalg.solve()` 解线性方程组 $3x + y = 9$、$x + 2y = 8$；用 NumPy 代入验证求得的解。
2. 使用 `scipy.optimize.root()` 求方程 $x^2 - 2 = 0$ 的正根和负根，分别给出初始猜测值。打印求根是否成功以及结果，并检查函数值是否接近 0。
3. 使用 `scipy.optimize.minimize()` 求 $f(x) = x^2 + x + 2$ 的最小值，报告极小点和函数值，并与解析结果比较。
4. 生成两组独立的模拟样本：各 100 个观测值，第一组来自均值 0、标准差 1 的正态分布，第二组来自均值 1、标准差 1 的正态分布。固定随机种子，使用 `scipy.stats.ttest_ind()` 进行独立样本 t 检验；报告 t 统计量、p 值及在 0.05 显著性水平下的判断。
5. 用 Matplotlib 为实验2的有效成绩画直方图，并另画箱线图；为函数 $y = x$、$y = x^2$、$y = x^3$ 在 $0 \le x \le 5$ 上画同一坐标系的折线图。每张图包含标题、坐标轴标签，折线图包含图例。直接在 Notebook 中显示图像，不保存为文件。

实验4：pandas Series、DataFrame 与内存中的 CSV 读写

操作任务：

1. 使用以下模拟数据创建 `DataFrame`，行索引为姓名；计算 `Total`（三科总分）和 `Average`（三科平均分）两列。输出行索引、列名、前 3 行和基本统计信息。

```
records = {
    "Name": ["Alice", "Bob", "Carol", "David", "Eva", "Frank"],
    "Group": ["A", "A", "B", "B", "A", "B"],
    "Math": [88, 72, 91, 65, 79, 84],
    "English": [90, 68, 85, 73, 92, 77],
    "Python": [95, 80, 89, 70, 86, 82],
}
```

2. 分别用 `loc` 和 `iloc` 读取 Carol 的记录，并筛选平均分不低于 80 的学生。

3. 分别用 `mean(axis=0)` 和 `mean(axis=1)` 计算三科成绩的列均值和每名学生的行均值。

4. 使用 `apply()` 计算每名学生的三科最高分，使用 `agg()` 一次得到每科的最小值、均值和最大值。任选一列使用 `rank()` 给学生排名。

5. 按 `Group` 分组，统计每组人数和三科平均分。再用 `transform('mean')` 为每行增加其所在组的 `Python` 平均分。

6. 用 `filter()` 保留 `Python` 平均分不低于 80 的组；输出保留的组和学生。

7. 计算三科成绩之间的协方差矩阵和皮尔森相关系数矩阵；选择其中一对科目，解释协方差符号与相关系数大小的含义。